SLIIT
Discover Your Future

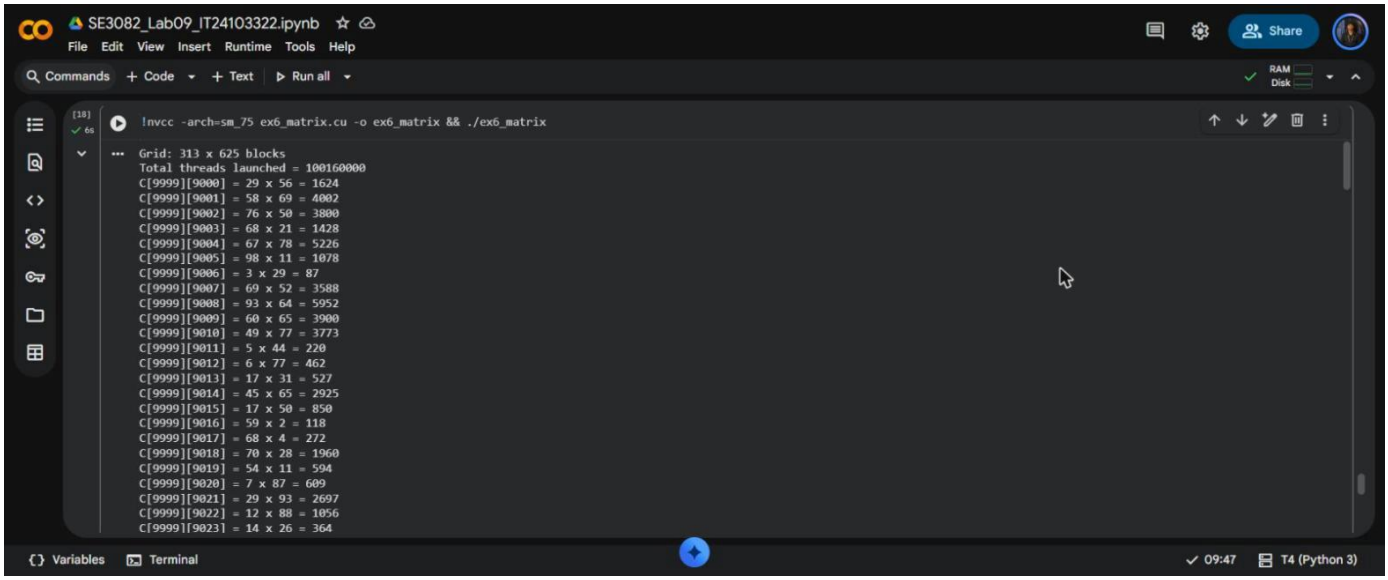
BSc (Hons) in Computer Science
Year 3

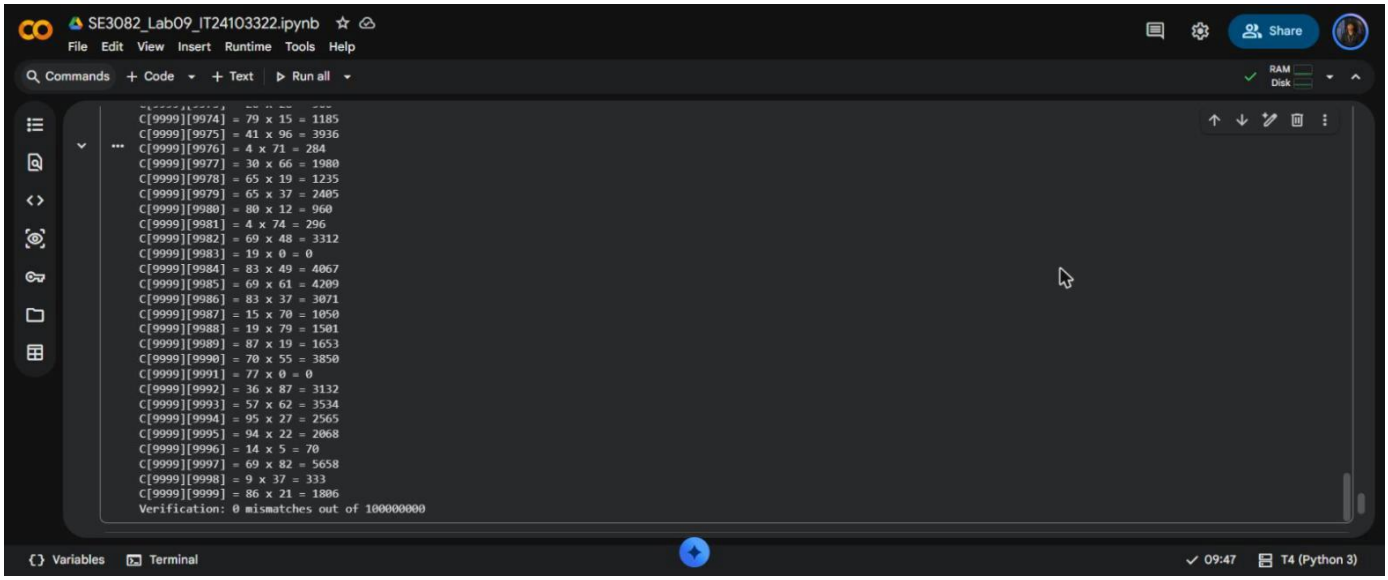
Lab Sheets

SE3082 – Parallel Computing **Semster 1, 2026**

IT24103322

Exercise 6





A, B and C are 10,000 × 10,000 matrices stored as flat row-major arrays allocated with malloc and cudaMalloc (400 MB each). Each thread computes one element, C[r][c] = A[r][c] × B[r][c]; this is an element-wise product, not matrix multiplication.

The block is dim3 threadsPerBlock(32, 16), 512 threads. The grid is rounded up in both directions: (10000 + 31) / 32 = 313 blocks across the columns and (10000 + 15) / 16 = 625 blocks down the rows. In the kernel col = blockIdx.x * blockDim.x + threadIdx.x and row = blockIdx.y * blockDim.y + threadIdx.y. The element is computed only if row < rows and col < cols, and the flat index is idx = row * cols + col. The last 1000 elements, C[9999][9000] to C[9999][9999], are printed.

Item	**Expected value**	**My result**
Grid	313 x 625 blocks	Grid: 313 x 625 blocks
Total threads launched	100,160,000	100160000
C[9999][9000]	29 x 56 = 1624	29 x 56 = 1624
C[9999][9999]	86 x 21 = 1806	86 x 21 = 1806
Verification	0 mismatches out of 100000000	0 mismatches out of 100000000

All the values match the self-check. 313 × 625 = 195,625 blocks of 512 threads launch 100,160,000 threads, 160,000 more than the 100,000,000 elements. The extra threads are in the last block column, where 16 of the 32 threads in x are past column 9999, so both parts of the guard are needed. The run took about 6 seconds, mostly for generating the random values and the copies of 1.2 GB on the CPU side.

The notebook SE3082_Lab09_IT24103322.ipynb in my Google Drive contains the working cells for every exercise with the outputs visible.